

MapReduce:

More Power, Less Code

This is an introduction to MapReduce as a framework for beginners in distributed computing, and a short tutorial on Hadoop, an open source MapReduce implementation. It is intended for anyone with significant experience in programming and a flair for distributed systems.

MapReduce is a framework, a pattern, and a programming paradigm that allows us to carry out computations over several terabytes of data in a matter of seconds. When it comes to massive-scale architecture and a huge amount of data, with built-in fault tolerance, there's nothing better than this. But when we come to define MapReduce programming, it is basically just a combination of two functions—a map function and a reduce function. This shows not just the amount of simplicity exposed by the framework in terms of the efforts of the programmer, but also the sheer power and flexibility of the code that runs under the hood.

It was Google that first introduced MapReduce as a framework. It is used to build indexes for Google Web searches! It handles many petabytes of data every day, where programs are executed on a large-scale cluster. You might not realise the actual increase in performance when processing a limited amount of data on a limited amount of machines, but if you do dream of becoming that big one day (and of course you do!), then this is the way to go.

However, this should not push you into thinking that MapReduce is effective only for large datasets and

data-intensive computations; what can be more important here is that programmers without any experience with parallel and distributed systems can easily use distributed resources, with the help of MapReduce programming.

Here I would like to point out that there is some difference between distributed computing and parallel computing. Although parallel computing is more of a modified form of distributed computing, parallel computing generally refers to a large number of processors sharing the same memory or the same disks, while distributed computing is increasingly referred to as a cluster of nodes, where each node is an independent unit with its own memory and disk. These could be any computer or laptop that you find lying around you. This is why the days of dedicated high-performance supercomputers are gone. These days, the computational power of carefully tuned and configured clusters of such computers can easily match, or even exceed, the performance of several supercomputers that you might have heard of. Another advantage of using such clusters is that they have a linear scalability curve, and you don't have to go about buying a bigger and better server to get increased performance.